Aufgabe 1:

- **Ziel:** Eine Datei ändern, die im end-Branch nicht verändert wurde.
- **Befehlsschritte:**

  . Branches prüfen:

  ➔ git branch -a

  . Lokalen end-Branch anlegen:

  ➔ git checkout -b end origin/end

  . Auf master bleiben, Datei ändern, committen:

  ➔ git checkout master
  ➔ nano questlog.md
  ➔ git add questlog.md
  ➔ git commit -m "Change questlog only on master"

  . end in master mergen:

  ➔ git merge end

- **Beobachtung:**
  Es gab keine Konflikte, da die Änderung in einer anderen Datei lag.

Aufgabe 2:

- **Ziel:** Eine Datei ändern, die auch im end-Branch verändert wurde, aber an einer anderen Stelle.
- **Befehlsschritte:**

  . Auf master bleiben, Datei ändern an anderer Stelle:

  ➔ nano rucksack.md
  ➔ git add rucksack.md
  ➔ git commit -m "Change rucksack at different line"

  . end in master mergen:

  ➔ git merge end

- **Beobachtung:**
  Kein Konflikt, da die Änderungen an unterschiedlichen Stellen lagen.

Aufgabe 3:

- **Ziel:** Wie in 2, aber die Änderung exakt an derselben Stelle wie im end-Branch.
- **Befehlsschritte:**

  . Auf master bleiben, Datei an derselben Stelle ändern (identisch zu end):

  ➔ nano rucksack.md
  ➔ git add rucksack.md
  ➔ git commit -m "Change rucksack identical to end"

  . end in master mergen:

  ➔ git merge end

- **Beobachtung:**

  . Fall 1: Die Änderung ist identisch, kein Konflikt.

  . Fall 2: Die Änderung ist unterschiedlich, es entsteht ein Merge-Konflikt, den wir manuell lösen.

Aufgabe 4:

- **Ziel:** end auf die Spitze von master setzen, bevor gemerged wird.
- **Befehlsschritte:**

  . end-Branch wechseln:

  ➔ git checkout end

  . end auf master bringen:

  ➔ git merge master

  . Zurück auf master:

  ➔ git checkout master

  . end in master mergen:

  ➔ git merge end

- **Beobachtung:**
  Jetzt gab es einen Fast-Forward-Merge, da end die neuesten Änderungen von master übernommen hat, bevor gemerged wurde.

1. Projektübersicht

Das „Cat-Cafe“-Projekt ist eine Java-Anwendung, die die Verwaltung eines Katzen-Cafés simuliert. Die Kernkomponenten sind eine Klasse „CatCafe“, die Katzen verwaltet, sowie ein „Tree“-Modell für eine hierarchische Darstellung. Das Projekt nutzt Gradle als Build-Tool, JUnit für Tests und Google Java Format für die Code-Formatierung. Die gesamte Anwendung wird über eine CI-Pipeline bei GitHub Actions automatisch gebaut, getestet und formatiert.

2. Gradle-Konfiguration

Das Projekt verwendet Gradle als Build-Tool. In der build.gradle-Datei sind die JUnit-Abhängigkeiten definiert (JUnit Jupiter 5.x), ebenso das Spotless-Plugin für die Code-Formatierung. Die mainClass ist auf die Klasse catcafe.Main gesetzt. Die Tests werden mit useJUnitPlatform() gestartet, und das Google Java Format sorgt für konsistente Formatierung.

3. Testfälle

Es wurden 10 JUnit-Tests für die Klasse CatCafe implementiert. Die Tests prüfen typische und Randfälle: Das Hinzufügen von Katzen, Entfernen, Kapazitätsgrenzen, Suche nach Katzen und Leeren des Cafés. Jedes Testpaar folgt dem „Given – When – Then“-Muster und deckt unterschiedliche Aspekte ab, z.B. die korrekte Behandlung von doppelten Einträgen oder unerwarteten Operationen.

4. CI-Pipeline

Die CI-Pipeline ist über GitHub Actions implementiert. Bei jedem Push auf den Hauptbranch wird das Projekt gebaut, die JUnit-Tests ausgeführt und die Formatierung mit Spotless geprüft. Somit wird sichergestellt, dass jeder Commit den Anforderungen entspricht.

5. Relevanz und Unterschiedlichkeit der Testfälle

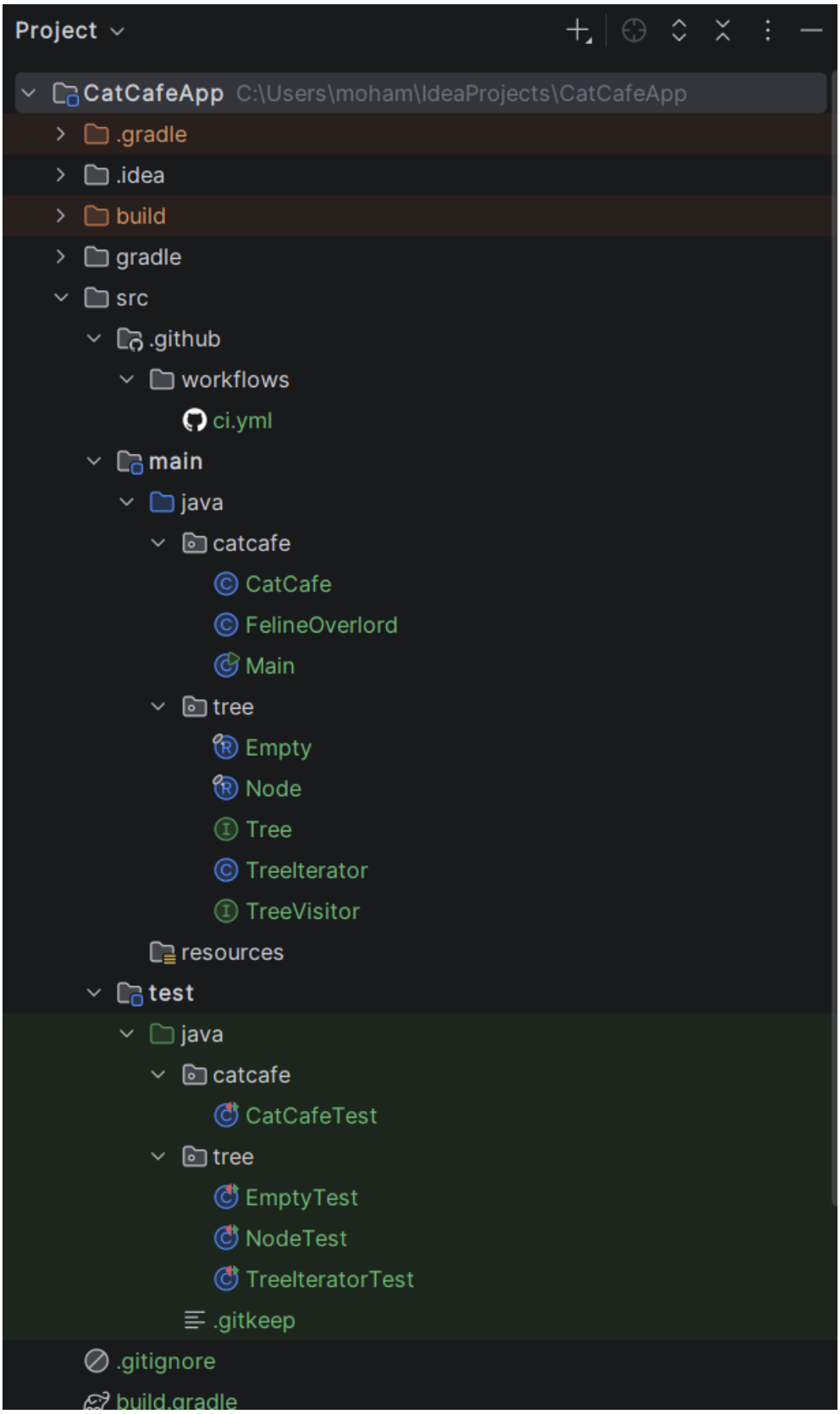
Die Testfälle sind relevant, da sie die Kernfunktionalitäten (Hinzufügen, Entfernen, Kapazität) prüfen und Randbedingungen abdecken, wie z.B. die Behandlung von doppelten Katzen. Sie unterscheiden sich durch verschiedene Ausgangssituationen (leeres Café, volle Kapazität) und prüfen sowohl Standard- als auch Fehlerfälle.

6. Versionierung und Remote

Das Projekt wurde in ein öffentliches GitHub-Repo hochgeladen. Lokales und Remote-Repo sind über Git verbunden, sodass alle Änderungen durch Push und Pull synchronisiert werden. Die CI-Pipeline läuft bei jedem Commit automatisch und sichert so die Konsistenz.

7. Fazit und Ausblick

Das Projekt ist stabil und erfüllt die Anforderungen. Zukünftige Erweiterungen könnten weitere Funktionen wie Preisberechnungen oder Reservierungen umfassen. Auch könnten weitere Tests und Refactorings eingebaut werden.



```
PS C:\Users\moham\IdeaProjects\CatCafeApp> ./gradlew clean test

BUILD SUCCESSFUL in 3s
4 actionable tasks: 4 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.4.0/userguide/configuration_cache_enabling.html

PS C:\Users\moham\IdeaProjects\CatCafeApp> ./gradlew clean build

BUILD SUCCESSFUL in 4s
11 actionable tasks: 11 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.4.0/userguide/configuration_cache_enabling.html

PS C:\Users\moham\IdeaProjects\CatCafeApp> ./gradlew clean run

> Task :run
Anzahl Katzen im Caf├®: 2

BUILD SUCCESSFUL in 1s
3 actionable tasks: 3 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.4.0/userguide/configuration_cache_enabling.html

PS C:\Users\moham\IdeaProjects\CatCafeApp> ./gradlew spotlessApply

BUILD SUCCESSFUL in 2s
3 actionable tasks: 3 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/9.4.0/userguide/configuration_cache_enabling.html

PS C:\Users\moham\IdeaProjects\CatCafeApp>
```